Cortex-M7 核心开发手册

Revision History

Draft Date	Revision No.	Description
2024/10/22	V1.0	1. 初始版本。

目　　录

前　言 4
1 Cortex-M7 核心案例开发说明 5
1.1　搭建 Cortex-M7 核心开发环境 5
1.2　Cortex-M7 核心案例编译 8
1.2.1　编译 TCM 版本程序镜像文件 9
1.2.2　编译 DDR 版本程序镜像文件 10
1.3　加载运行 Cortex-M7 核心程序镜像文件 11
1.3.1　运行 TCM 版本程序镜像文件 12
1.3.2　运行 DDR 版本程序镜像文件 13
1.4　设备树文件说明 14
2 Baremetal-demos 14
2.1　led_flash 案例 14
2.1.1　案例说明 14
2.1.2　案例测试 15
2.1.3　关键代码 15
2.2　uart_echo 案例 16
2.2.1　案例说明 16
2.2.2　案例测试 17
2.2.3　关键代码 17
3 RTOS-demos 18
3.1　led_flash 案例 18
3.1.1　案例说明 18
3.1.2　案例测试 19
3.1.3　关键代码 19
3.2　uart_echo 案例 21
3.2.1　案例说明 21
3.2.2　案例测试 21
3.2.3　关键代码 22

更多帮助 24

前 言

本文档适用开发环境：

Windows 开发环境：Windows 7 64bit、Windows 10 64bit

虚拟机：VMware15.5.5

Linux 开发环境：Ubuntu 20.04.6 64bit

U-Boot：U-Boot-2022.04

Kernel：Linux-5.15.71-rt51

SDK 开发包：SDK_2_12_0_MIMX8ML8xxxKZ

交叉编译工具链：gcc-arm-none-eabi-10.3-2021.10-x86_64-linux

i.MX 8M Plus 处理器的 Cortex-M7 核心支持运行裸机程序和 FreeRTOS 系统，本文主要演示 Cortex-M7 核心的裸机和 FreeRTOS 开发案例。

Cortex-M7 核心案例位于“4-软件资料\Demo\”目录下，主要包含 Baremetal-demos（Cortex-M7 核心裸机案例）、RTOS-demos（Cortex-M7 核心 FreeRTOS 案例）。

案例具体说明如下。

表 1

<table>
<tr><th colspan="4">目录/文件</th><th>说明</th></tr>
<tr><td rowspan="5">Baremetal-demos、RTOS-demos</td><td rowspan="3">led_flash</td><td rowspan="2">bin</td><td>led_flash.bin</td><td>TCM 镜像</td></tr>
<tr><td>led_flash_ddr.bin</td><td>DDR 镜像</td></tr>
<tr><td colspan="2">project</td><td>程序源码</td></tr>
<tr><td rowspan="3">uart_echo</td><td rowspan="2">bin</td><td>uart_echo.bin</td><td>TCM 镜像</td></tr>
<tr><td>uart_echo_ddr.bin</td><td>DDR 镜像</td></tr>
<tr><td colspan="2">project</td><td>程序源码</td></tr>
</table>

备注：由于 TLIMX8MP-EVM 评估板未引出 JTAG 仿真调试接口，因此不支持 JTAG 仿真调试，如需调试建议使用串口进行调试。

1 Cortex-M7 核心案例开发说明

本章节以 Baremetal-demos 目录下的 led_flash 案例为例，演示 Cortex-M7 核心案例的编译、加载运行方法，请将 led_flash 案例拷贝至 Ubuntu 工作目录。

```
tronlong@tronlong-virtual-machine:~/Demo/Baremetal-demos/led_flash$ tree
.
├── bin
│   ├── led_flash.bin
│   └── led_flash_ddr.bin
└── project
    ├── armgcc
    │   ├── armgcc.cmake
    │   ├── build_ddr_debug.sh
    │   ├── build_ddr_release.sh
    │   ├── build_debug.sh
    │   ├── build_release.sh
    │   ├── clean.sh
    │   ├── CMakeLists.txt
    │   ├── config.cmake
    │   ├── flags.cmake
    │   ├── Makefile
    │   ├── MIMX8ML8xxxxx_cm7_ddr_ram.ld
    │   └── MIMX8ML8xxxxx_cm7_ram.ld
    ├── board.c
    ├── board.h
    ├── clock_config.c
    ├── clock_config.h
    ├── empty_rsc_table.c
    ├── fsl_iomuxc.h
    ├── led_flash.c
    ├── pin_mux.c
    └── pin_mux.h

3 directories, 23 files
tronlong@tronlong-virtual-machine:~/Demo/Baremetal-demos/led_flash$
```

图 1

1.1 搭建 Cortex-M7 核心开发环境

（1） 安装 SDK 开发包

Cortex-M7 核心使用的 SDK 开发包为产品资料“4-软件资料\Tools\SDK\”目录下的 SDK_2_12_0_MIMX8ML8xxxKZ.tar.gz。请将 SDK 开发包拷贝至 Ubuntu 共享目录，进入 Ubuntu 执行如下命令创建 SDK_2_12_0_MIMX8ML8xxxKZ 目录，并将 SDK 开发包解压至该目录下。

Host# mkdir /home/tronlong/SDK_2_12_0_MIMX8ML8xxxKZ

Host# tar -zxf /mnt/hgfs/SharedFolders/SDK_2_12_0_MIMX8ML8xxxKZ.tar.gz -C /home/tronlong/SDK_2_12_0_MIMX8ML8xxxKZ/

```
tronlong@tronlong-virtual-machine:~$ mkdir /home/tronlong/SDK_2_12_0_MIMX8ML8xxxKZ
tronlong@tronlong-virtual-machine:~$ tar -zxf /mnt/hgfs/SharedFolders/SDK_2_12_0_MIMX8ML8xxxKZ.tar.gz -C /
home/tronlong/SDK_2_12_0_MIMX8ML8xxxKZ/
tronlong@tronlong-virtual-machine:~$ ls SDK_2_12_0_MIMX8ML8xxxKZ/
boards      COPYING-BSD-3  LA_OPT_NXP_Software_License.txt   rtos
CMSIS       devices        middleware                         SW-Content-Register.txt
components  docs           MIMX8ML8xxxKZ_manifest_v3_10.xml   tools
tronlong@tronlong-virtual-machine:~$
```

图 2

官方参考案例位于"SDK_2_12_0_MIMX8ML8xxxKZ/boards/evkmimx8mp/"目录下，包括 demo_apps、driver_examples、cmsis_driver_examples、multicore_examples 和 rtos_examples，如下所示。

```
tronlong@tronlong-virtual-machine:~/SDK_2_12_0_MIMX8ML8xxxKZ/boards/evkmimx8mp$ ls
cmsis_driver_examples  driver_examples  multicore_examples  rtos_examples
demo_apps              evkmimx8mp.png   project_template
tronlong@tronlong-virtual-machine:~/SDK_2_12_0_MIMX8ML8xxxKZ/boards/evkmimx8mp$ tree -L 1 demo_apps/
demo_apps/
├── hello_world
├── low_power_voice
├── low_power_wakeword
└── sai_low_power_audio

4 directories, 0 files
tronlong@tronlong-virtual-machine:~/SDK_2_12_0_MIMX8ML8xxxKZ/boards/evkmimx8mp$ tree -L 1 driver_examples/
driver_examples/
├── asrc
├── canfd
├── ecspi
├── flexcan
├── gpio
├── gpt
├── i2c
├── pdm
├── pwm
├── rdc
├── sai
├── sdma
├── sema4
├── tmu
├── uart
└── wdog

16 directories, 0 files
tronlong@tronlong-virtual-machine:~/SDK_2_12_0_MIMX8ML8xxxKZ/boards/evkmimx8mp$
```

图 3

```
tronlong@tronlong-virtual-machine:~/SDK_2_12_0_MIMX8ML8xxxKZ/boards/evkmimx8mp$ tree -L 1 cmsis_driver_exa
mples/
cmsis_driver_examples/
├── ecspi
├── i2c
└── uart

3 directories, 0 files
tronlong@tronlong-virtual-machine:~/SDK_2_12_0_MIMX8ML8xxxKZ/boards/evkmimx8mp$ tree -L 1 multicore_exampl
es/
multicore_examples/
├── rpmsg_lite_pingpong_rtos
└── rpmsg_lite_str_echo_rtos

2 directories, 0 files
tronlong@tronlong-virtual-machine:~/SDK_2_12_0_MIMX8ML8xxxKZ/boards/evkmimx8mp$ tree -L 1 rtos_examples/
rtos_examples/
├── freertos_ecspi
├── freertos_event
├── freertos_generic
├── freertos_hello
├── freertos_i2c
├── freertos_mutex
├── freertos_queue
├── freertos_sem
├── freertos_swtimer
├── freertos_tickless
└── freertos_uart

11 directories, 0 files
tronlong@tronlong-virtual-machine:~/SDK_2_12_0_MIMX8ML8xxxKZ/boards/evkmimx8mp$
```

图 4

（2） 交叉编译工具链配置

将产品资料“4-软件资料\Tools\SDK\”目录下的交叉编译工具链压缩包 gcc-arm-none-eabi-10.3-2021.10-x86_64-linux.tar.bz2 拷贝至 Ubuntu 共享目录，再执行如下命令将其解压，解压完成后将生成 gcc-arm-none-eabi-10.3-2021.10 目录。

Host# tar -jxf /mnt/hgfs/SharedFolders/gcc-arm-none-eabi-10.3-2021.10-x86_64-linux.tar.bz2 -C /home/tronlong/

```
tronlong@tronlong-virtual-machine:~$ tar -jxf /mnt/hgfs/SharedFolders/gcc-arm-none-eabi-10.3-2021.10-x86_64-linux
.tar.bz2 -C /home/tronlong/
tronlong@tronlong-virtual-machine:~$ ls gcc-arm-none-eabi-10.3-2021.10/ -l
total 16
drwxr-xr-x 6 tronlong tronlong 4096 10月 18  2021 arm-none-eabi
drwxr-xr-x 2 tronlong tronlong 4096 10月 18  2021 bin
drwxr-xr-x 3 tronlong tronlong 4096 10月 18  2021 lib
drwxr-xr-x 4 tronlong tronlong 4096 10月 18  2021 share
tronlong@tronlong-virtual-machine:~$
```

图 5

在 Ubuntu 执行如下命令，分别设置编译相关的环境和 SDK 开发包路径。

Host# export ARMGCC_DIR=/home/tronlong/gcc-arm-none-eabi-10.3-2021.10

Host# export PATH=$PATH:/home/tronlong/gcc-arm-none-eabi-10.3-2021.10/bin

Host# export SDKPath=/home/tronlong/SDK_2_12_0_MIMX8ML8xxxKZ/

```
tronlong@tronlong-virtual-machine:~$ export ARMGCC_DIR=/home/tronlong/gcc-arm-none-eabi-10.3-2021.10
tronlong@tronlong-virtual-machine:~$ export PATH=$PATH:/home/tronlong/gcc-arm-none-eabi-10.3-2021.10/bin
tronlong@tronlong-virtual-machine:~$ export SDKPath=/home/tronlong/SDK_2_12_0_MIMX8ML8xxxKZ/
tronlong@tronlong-virtual-machine:~$
```

图 6

（3） 安装 cmake 工具

Host# sudo apt-get install cmake

备注：cmake 版本需 3.0.0 及以上。

```
tronlong@tronlong-virtual-machine:~$ sudo apt-get install cmake
[sudo] password for tronlong:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
  bridge-utils ubuntu-fan
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu cmake-data gcc gcc-9 libasan5 libbinutils libc-dev-bin
  libc6-dev libcrypt-dev libctf-nobfd0 libctf0 libgcc-9-dev libitm1 libjsoncpp1 liblsan0 libquadmath0 librhash0
  libtsan0 libubsan1 linux-libc-dev make manpages-dev
Suggested packages:
  binutils-doc cmake-doc ninja-build gcc-multilib autoconf automake libtool flex bison gcc-doc gcc-9-multilib
  gcc-9-doc gcc-9-locales glibc-doc make-doc
The following NEW packages will be installed:
  binutils binutils-common binutils-x86-64-linux-gnu cmake cmake-data gcc gcc-9 libasan5 libbinutils
  libc-dev-bin libc6-dev libcrypt-dev libctf-nobfd0 libctf0 libgcc-9-dev libitm1 libjsoncpp1 liblsan0
  libquadmath0 librhash0 libtsan0 libubsan1 linux-libc-dev make manpages-dev
0 upgraded, 25 newly installed, 0 to remove and 61 not upgraded.
Need to get 31.3 MB of archives.
After this operation, 149 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
```

图 7

```
Setting up cmake (3.16.3-1ubuntu1.20.04.1) ...
Setting up libc6-dev:amd64 (2.31-0ubuntu9.16) ...
Setting up binutils-x86-64-linux-gnu (2.34-6ubuntu1.9) ...
Setting up binutils (2.34-6ubuntu1.9) ...
Setting up gcc-9 (9.4.0-1ubuntu1~20.04.2) ...
Setting up gcc (4:9.3.0-1ubuntu2) ...
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for libc-bin (2.31-0ubuntu9.16) ...
tronlong@tronlong-virtual-machine:~$
tronlong@tronlong-virtual-machine:~$ cmake --version
cmake version 3.16.3

CMake suite maintained and supported by Kitware (kitware.com/cmake).
tronlong@tronlong-virtual-machine:~$
```

图 8

1.2 Cortex-M7 核心案例编译

我司提供的 Cortex-M7 核心案例提供了多个编译脚本（位于案例"project/armgcc/"目录下），可分别编译生成在 TCM 和 DDR 上运行的程序镜像文件，具体说明如下。

表 2

编译脚本	说明
build_debug.sh	编译生成在 TCM 上运行的 debug 版本程序镜像文件
build_release.sh	编译生成在 TCM 上运行的 release 版本程序镜像文件
build_ddr_debug.sh	编译生成在 DDR 上运行的 debug 版本程序镜像文件
build_ddr_release.sh	编译生成在 DDR 上运行的 release 版本程序镜像文件

我司默认使用 xxx_debug.sh 编译脚本，即编译生成在 TCM 和 DDR 上运行的 debug 版本程序镜像文件。

1.2.1 编译 TCM 版本程序镜像文件

进入案例"project/armgcc/"目录，执行如下命令运行 build_debug.sh 编译脚本，编译生成在 TCM 上运行的程序镜像文件。

Host# ./build_debug.sh

```
tronlong@tronlong-virtual-machine:~/Demo/Baremetal-demos/led_flash/project/armgcc$ ls
armgcc.cmake          build_debug.sh    CMakeLists.txt  Makefile
build_ddr_debug.sh    build_release.sh  config.cmake    MIMX8ML8xxxxx_cm7_ddr_ram.ld
build_ddr_release.sh  clean.sh          flags.cmake     MIMX8ML8xxxxx_cm7_ram.ld
tronlong@tronlong-virtual-machine:~/Demo/Baremetal-demos/led_flash/project/armgcc$ ./build_debug.sh
-- TOOLCHAIN_DIR: /home/tronlong/gcc-arm-none-eabi-10.3-2021.10
-- BUILD_TYPE: debug
-- TOOLCHAIN_DIR: /home/tronlong/gcc-arm-none-eabi-10.3-2021.10
-- BUILD_TYPE: debug
-- The ASM compiler identification is GNU
-- Found assembler: /home/tronlong/gcc-arm-none-eabi-10.3-2021.10/bin/arm-none-eabi-gcc
-- The C compiler identification is GNU 10.3.1
-- The CXX compiler identification is GNU 10.3.1
utility_debug_console_lite component is included.
component_iuart_adapter component is included.
driver_common component is included.
driver_clock component is included.
device_MIMX8ML8_CMSIS component is included.
CMSIS_Include_core_cm component is included.
driver_iuart component is included.
utility_assert_lite component is included.
driver_igpio component is included.
driver_rdc component is included.
driver_audiomix component is included.
component_lists component is included.
device_MIMX8ML8_startup component is included.
device_MIMX8ML8_system component is included.
utilities_misc_utilities component is included.
-- Configuring done
-- Generating done
-- Build files have been written to: /home/tronlong/Demo/Baremetal-demos/led_flash/project/armgcc
Scanning dependencies of target led_flash.elf
```

图 9

编译完成后，将在当前路径下生成 debug 目录，并在 debug 目录下生成 xxx.bin 和 xxx.elf 程序镜像文件。

```
[ 95%] Building C object CMakeFiles/led_flash.elf.dir/home/tronlong/SDK_2_12_0_MIMX8ML8xxxKZ/devices/MIMX8ML8/uti
lities/fsl_sbrk.c.obj
[100%] Linking C executable debug/led_flash.elf
Memory region         Used Size  Region Size  %age Used
    m_interrupts:         680 B         1 KB     66.41%
          m_text:       16860 B       127 KB     12.96%
          m_data:        2224 B       128 KB      1.70%
         m_data2:          0 GB        16 MB      0.00%
[100%] Built target led_flash.elf
tronlong@tronlong-virtual-machine:~/Demo/Baremetal-demos/led_flash/project/armgcc$ ls debug/
led_flash.bin  led_flash.elf
tronlong@tronlong-virtual-machine:~/Demo/Baremetal-demos/led_flash/project/armgcc$
```

图 10

1.2.2 编译 DDR 版本程序镜像文件

执行如下命令运行 build_ddr_debug.sh 编译脚本，编译生成在 DDR 上运行的程序镜像文件。

Host# ./build_ddr_debug.sh

```
tronlong@tronlong-virtual-machine:~/Demo/Baremetal-demos/led_flash/project/armgcc$ ./build_ddr_debug.sh
-- TOOLCHAIN_DIR: /home/tronlong/gcc-arm-none-eabi-10.3-2021.10
-- BUILD_TYPE: ddr_debug
-- TOOLCHAIN_DIR: /home/tronlong/gcc-arm-none-eabi-10.3-2021.10
-- BUILD_TYPE: ddr_debug
-- The ASM compiler identification is GNU
-- Found assembler: /home/tronlong/gcc-arm-none-eabi-10.3-2021.10/bin/arm-none-eabi-gcc
-- The C compiler identification is GNU 10.3.1
-- The CXX compiler identification is GNU 10.3.1
utility_debug_console_lite component is included.
component_iuart_adapter component is included.
driver_common component is included.
driver_clock component is included.
device_MIMX8ML8_CMSIS component is included.
CMSIS_Include_core_cm component is included.
driver_iuart component is included.
utility_assert_lite component is included.
driver_igpio component is included.
driver_rdc component is included.
driver_audiomix component is included.
component_lists component is included.
device_MIMX8ML8_startup component is included.
device_MIMX8ML8_system component is included.
utilities_misc_utilities component is included.
-- Configuring done
-- Generating done
-- Build files have been written to: /home/tronlong/Demo/Baremetal-demos/led_flash/project/armgcc
Scanning dependencies of target led_flash.elf
[  4%] Building C object CMakeFiles/led_flash.elf.dir/home/tronlong/Demo/Baremetal-demos/led_flash/project/clock_
config.c.obj
[  9%] Building C object CMakeFiles/led_flash.elf.dir/home/tronlong/Demo/Baremetal-demos/led_flash/project/board.
c.obj
[ 14%] Building C object CMakeFiles/led_flash.elf.dir/home/tronlong/Demo/Baremetal-demos/led_flash/project/pin_mu
x.c.obj
```

图 11

编译完成后，将在当前路径下生成 ddr_debug 目录，并在 ddr_ddebug 目录下生成 xxx.bin 和 xxx.elf 程序镜像文件。

```
[ 95%] Building C object CMakeFiles/led_flash.elf.dir/home/tronlong/SDK_2_12_0_MIMX8ML8xxxKZ/devices/MIMX8ML8/sys
tem_MIMX8ML8_cm7.c.obj
[100%] Linking C executable ddr_debug/led_flash.elf
Memory region         Used Size  Region Size  %age Used
    m_interrupts:         680 B         1 KB     66.41%
          m_text:       16860 B      2047 KB      0.80%
          m_data:        2224 B         2 MB      0.11%
         m_data2:          0 GB        12 MB      0.00%
[100%] Built target led_flash.elf
tronlong@tronlong-virtual-machine:~/Demo/Baremetal-demos/led_flash/project/armgcc$ ls ddr_debug/
led_flash.bin  led_flash.elf
tronlong@tronlong-virtual-machine:~/Demo/Baremetal-demos/led_flash/project/armgcc$
```

图 12

1.3 加载运行 Cortex-M7 核心程序镜像文件

请将编译生成的 TCM 或 DDR 版本的 xxx.bin 程序镜像文件重命名为 m7_app.bin，并将其拷贝至 Linux 系统启动卡 BOOT 分区中。

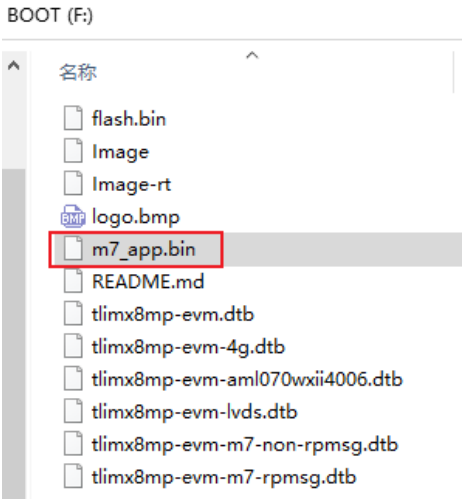

图 13

请将 Linux 系统启动卡 BOOT 分区中的支持 Cortex-M7 核心的 tlimx8mp-evm-m7-non-rpmsg.dtb 设备树复制并重命名为 tlimx8mp-evm.dtb，替换分区中原来默认的 tlimx8mp-evm.dtb 设备树。

备注：设备树文件替换具体说明请参考设备树文件说明章节。

将 Linux 系统启动卡插至评估板 Micro SD 卡槽，启动方式选择拨码开关拨至 1100(1~4)，此档位为 Micro SD 卡启动模式。使用 USB 串口线将评估板的 USB TO UART2 串口连接至 PC 机的 USB 接口，使用 RS232 交叉串口母母线、USB 转 RS232 公头串口线将评估板的 RS232 UART4 串口连接至 PC 机的 USB 接口，如下图所示。

其中 USB TO UART2 作为 Cortex-A53 调试串口，RS232 UART4 作为 Cortex-M7 核心调试串口。

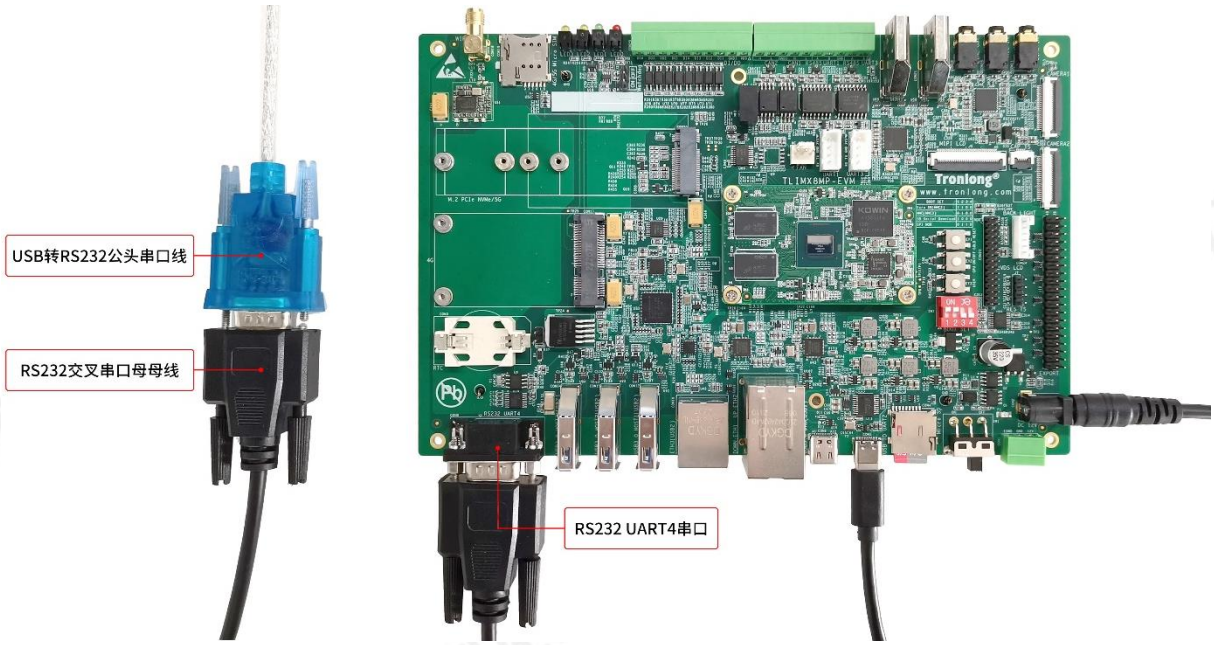

图 14

1.3.1 运行 TCM 版本程序镜像文件

系统默认是 TCM 版本 Cortex-M7 核心程序镜像文件的运行环境，评估板上电启动，在 U-Boot 程序运行时，评估板 USER1 LED 以 500ms 间隔进行闪烁，同时 RS232 UART4 调试串口打印如下类似信息。

```
GPIO Driver example
The LED is blinking.
```

图 15

1.3.2 运行 DDR 版本程序镜像文件

评估板上电启动，在 U-Boot 倒计时结束之前按下空格键进入 U-Boot 命令行模式，在 U-Boot 命令行执行如下命令，设置 DDR 版本的 Cortex-M7 核心程序镜像文件运行环境。

U-Boot=> setenv m7_run_mem DDR

U-Boot=> saveenv

U-Boot=> reset

```
u-boot=>
u-boot=> setenv m7_run_mem DDR
u-boot=> saveenv
Saving Environment to MMC... Writing to MMC(2)... OK
u-boot=> reset
```

图 16

评估板重启，在 U-Boot 程序运行时，评估板 USER1 LED 以 500ms 间隔进行闪烁，同时 RS232 UART4 调试串口打印如下类似信息。

```
GPIO Driver example
The LED is blinking.
```

图 17

如需恢复默认的 TCM 版本 Cortex-M7 核心程序镜像文件运行环境，请执行如下命令。

U-Boot# setenv m7_run_mem TCM

U-Boot# saveenv

U-Boot# reset

```
u-boot=>
u-boot=> setenv m7_run_mem TCM
u-boot=> saveenv
Saving Environment to MMC... Writing to MMC(2)... OK
u-boot=> reset
```

图 18

1.4 设备树文件说明

由于 Cortex-M7 核心内部默认不加载 Cortex-M7 核心程序，使能 Cortex-M7 核心程序未使用的外设将会与 Cortex-A53 核心发生冲突，因此 Cortex-M7 核心设备树文件在基础设备树文件 tlimx8mp-evm.dts 的基础上将未使用的外设进行禁用。

```
&ecspi2 {
        status = "disabled";
};

&flexcan1 {
        status = "disabled";
};

&flexspi {
        status = "disabled";
};

&i2c3 {
        status = "disabled";
};

&pwm4{
        status = "disabled";
};

&sai3 {
        status = "disabled";
};

&micfil {
        status = "disabled";
};

&sdma3{
        status = "disabled";
};

&uart3 {
    status = "disabled";
};

&uart4 {
        status = "disabled";
};

```

图 19 tlimx8mp-evm-m7-non-rpmsg.dts

2 Baremetal-demos

2.1 led_flash 案例

2.1.1 案例说明

案例功能：Cortex-M7 核心控制 GPIO 输出，让 USER LED1 间隔 500ms 闪烁。

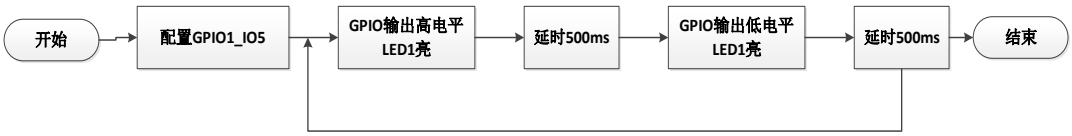

图 20

2.1.2 案例测试

案例 bin 目录下分别提供了 TCM 版本程序镜像文件 led_flash.bin 和 DDR 版本程序镜像文件 led_flash_ddr.bin。

请参考“加载运行 Cortex-M7 核心程序镜像文件”章节运行镜像文件。程序运行后，评估板 USER1 LED 以 500ms 间隔进行闪烁，同时 RS232 UART4 调试串口打印如下类似信息。

```
GPIO Driver example
The LED is blinking.
```

图 21

2.1.3 关键代码

程序主体位于 main 函数中，定义位于 led_flash.c 文件中。

（1） 定义 GPIO 结构体。

```
/* Define the init structure for the output LED pin*/
gpio_pin_config_t led_config = {kGPIO_DigitalOutput, 0, kGPIO_NoIntmode};
```

图 22

（2） 初始化内存、资源控制器、GPIO、系统时钟和调试串口。

```
/* Board pin, clock, debug console init */
/* M7 has its local cache and enabled by default,
 * need to set smart subsystems (0x28000000 ~ 0x3FFFFFFF)
 * non-cacheable before accessing this address region */
BOARD_InitMemory();

/* Board specific RDC settings */
BOARD_RdcInit();

BOARD_InitBootPins();
BOARD_BootClockRUN();
BOARD_InitDebugConsole();
```

图 23

（3） 串口打印信息。

```
/* Print a note to terminal. */
PRINTF("\r\n GPIO Driver example\r\n");
PRINTF("\r\n The LED is blinking.\r\n");
```

图 24

（4） 初始化 LED 引脚，设置为 Output。

```
/* Init output LED GPIO. */
GPIO_PinInit(EXAMPLE_LED_GPIO, EXAMPLE_LED_GPIO_PIN, &led_config);
```

图 25

（5） 配置 LED 进行闪烁，间隔时间 500ms。

```
while (1)
{
/* Gpio output low level, turn off the LED */
GPIO_PinWrite(EXAMPLE_LED_GPIO, EXAMPLE_LED_GPIO_PIN, 0U);

/* Keep the LED off at least for 500 ms */
SDK_DelayAtLeastUs(500000, SDK_DEVICE_MAXIMUM_CPU_CLOCK_FREQUENCY);

/* Gpio output high level, turn on the LED */
GPIO_PinWrite(EXAMPLE_LED_GPIO, EXAMPLE_LED_GPIO_PIN, 1U);

/* Keep the LED on at least for 500 ms */
SDK_DelayAtLeastUs(500000, SDK_DEVICE_MAXIMUM_CPU_CLOCK_FREQUENCY);
}
```

图 26

2.2 uart_echo 案例

2.2.1 案例说明

案例功能：Cortex-M7 核心使用 RS232 UART4 收发功能，实现每输入 8 个字符，在串口终端上回显输入的 8 个字符。

图 27

2.2.2 案例测试

案例 bin 目录下分别提供了 TCM 版本程序镜像文件 uart_echo.bin 和 DDR 版本程序镜像文件 uart_echo_ddr.bin。

请参考“加载运行 Cortex-M7 核心程序镜像文件”章节运行镜像文件。程序运行后，在 RS232 UART4 串口终端每输入 8 个字符后，串口终端将进行回显。

```
Uart interrupt transfer example
Board receives 8 characters then sends them out
Now please input:
tronlong
```

图 28

2.2.3 关键代码

程序主体位于 main 函数中，定义位于 uart_echo.c 文件中。

（1） 初始化内存、资源控制器、GPIO 和系统时钟。

```
/* M7 has its local cache and enabled by default,
 * need to set smart subsystems (0x28000000 ~ 0x3FFFFFFF)
 * non-cacheable before accessing this address region */
BOARD_InitMemory();

/* Board specific RDC settings */
BOARD_RdcInit();

BOARD_InitBootPins();
BOARD_BootClockRUN();
```

图 29

（2） 初始化串口。

```
/*
 * config.baudRate_Bps = 115200U;
 * config.parityMode = kUART_ParityDisabled;
 * config.dataBitsCount = kUART_EightDataBits;
 * config.stopBitCount = kUART_OneStopBit;
 * config.txFifoWatermark = 2;
 * config.rxFifoWatermark = 16;
 * config.enableTx = false;
 * config.enableRx = false;
 */
UART_GetDefaultConfig(&config);
config.baudRate_Bps     = BOARD_DEBUG_UART_BAUDRATE;
config.rxFifoWatermark = 16;
config.enableTx         = true;
config.enableRx         = true;
```

图 30

（3） 非阻塞接收 API，接收前判断 RX 空闲和 RX 缓冲是否为空。

```
/* If RX is idle and g_rxBuffer is empty, start to read data to g_rxBuffer. */
if ((!rxOnGoing) && rxBufferEmpty)
{
    rxOnGoing = true;
    UART_TransferReceiveNonBlocking(DEMO_UART, &g_uartHandle, &receiveXfer, NULL);
}
```

图 31

（4） 非阻塞发送 API，发送前判断 TX 空闲和 TX 缓冲是否为满。

```
/* If TX is idle and g_txBuffer is full, start to send data. */
if ((!txOnGoing) && txBufferFull)
{
    txOnGoing = true;
    UART_TransferSendNonBlocking(DEMO_UART, &g_uartHandle, &sendXfer);
}
```

图 32

3 RTOS-demos

3.1 led_flash 案例

3.1.1 案例说明

案例功能：Cortex-M7 核心控制 GPIO 输出，让 USER LED1 间隔 500ms 闪烁。

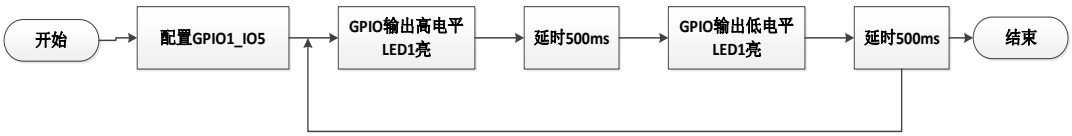

图 33

3.1.2 案例测试

案例 bin 目录下分别提供了 TCM 版本程序镜像文件 led_flash.bin 和 DDR 版本程序镜像文件 led_flash_ddr.bin。

请参考“加载运行 Cortex-M7 核心程序镜像文件”章节运行镜像文件。程序运行后，评估板 USER1 LED 以 500ms 间隔进行闪烁，同时 RS232 UART4 调试串口打印如下类似信息。

```
FreeRTOS GPIO Driver example
The LED is blinking.
```

图 34

3.1.3 关键代码

（1） 定义 GPIO 结构体。

```
/* Define the init structure for the output LED pin*/
gpio_pin_config_t led_config = {kGPIO_DigitalOutput, 0, kGPIO_NoIntmode};
```

图 35

（2） 初始化内存、资源控制器、GPIO、系统时钟和调试串口。

```
/* Init board hardware. */
/* M7 has its local cache and enabled by default,
 * need to set smart subsystems (0x28000000 ~ 0x3FFFFFFF)
 * non-cacheable before accessing this address region */
BOARD_InitMemory();

/* Board specific RDC settings */
BOARD_RdcInit();

BOARD_InitBootPins();
BOARD_BootClockRUN();
BOARD_InitDebugConsole();
```

图 36

（3） 串口打印信息。

```
/* Print a note to terminal. */
PRINTF("\r\n FreeRTOS GPIO Driver example\r\n");
PRINTF("\r\n The LED is blinking.\r\n");
```

图 37

（4） 初始化 LED 引脚，设置为 Output。

```
/* Init output LED GPIO. */
GPIO_PinInit(EXAMPLE_LED_GPIO, EXAMPLE_LED_GPIO_PIN, &led_config);
```

图 38

（5） 创建 led_flash 任务，创建成功则开启任务队列，否则打印失败提示后进入死循环。

```
/* Create the led_flash task. */
if (xTaskCreate(led_flash_task, "Led_flash_task", configMINIMAL_STACK_SIZE + 100, NULL, led_flash_task_PRIORITY, NULL) != pdPASS)
{
    PRINTF("Task creation failed!.\r\n");
    while (1)
        ;
}
vTaskStartScheduler();
for (;;)
    ;
```

图 39

（6） led_flash 任务启动后，任务进入死循环，配置 LED 进行闪烁，间隔时间 500ms。

```c
/*!
 * @brief Task switch the user-leds output status every 500ms.
 */
static void led_flash_task(void *pvParameters)
{
    while(1)
    {
        /* Gpio output low level, turn off the LED */
        GPIO_PinWrite(EXAMPLE_LED_GPIO, EXAMPLE_LED_GPIO_PIN, 0U);

        /* Keep the LED off at least for 500 ms */
        SDK_DelayAtLeastUs(500000, SDK_DEVICE_MAXIMUM_CPU_CLOCK_FREQUENCY);

        /* Gpio output high level, turn on the LED */
        GPIO_PinWrite(EXAMPLE_LED_GPIO, EXAMPLE_LED_GPIO_PIN, 1U);

        /* Keep the LED on at least for 500 ms */
        SDK_DelayAtLeastUs(500000, SDK_DEVICE_MAXIMUM_CPU_CLOCK_FREQUENCY);
    }
}
```

图 40

3.2 uart_echo 案例

3.2.1 案例说明

案例功能：Cortex-M7 核心使用 RS232 UART4 收发功能，实现每输入 8 个字符，在串口终端上回显输入的 8 个字符。

图 41

3.2.2 案例测试

案例 bin 目录下分别提供了 TCM 版本程序镜像文件 uart_echo.bin 和 DDR 版本程序镜像文件 uart_echo_ddr.bin。

请参考“加载运行 Cortex-M7 核心程序镜像文件”章节运行镜像文件。程序运行后，在 RS232 UART4 串口终端每输入 8 个字符后，串口终端将进行回显。

```
FreeRTOS UART driver example!
test
```

图 42

3.2.3 关键代码

程序主体位于 main 函数中，定义位于 uart_echo.c 文件中。

（1） 设置串口相关配置：波特率为 115200、关闭奇偶校验位、设置 1 位停止位。

```
uart_rtos_config_t uart_config = {
    .baudrate    = 115200,
    .parity      = kUART_ParityDisabled,
    .stopbits    = kUART_OneStopBit,
    .buffer      = background_buffer,
    .buffer_size = sizeof(background_buffer),
};
```

图 43

（2） 初始化内存、资源控制器、GPIO 和系统时钟。

```
/* Init board hardware. */
/* M7 has its local cache and enabled by default,
 * need to set smart subsystems (0x28000000 ~ 0x3FFFFFFF)
 * non-cacheable before accessing this address region */
BOARD_InitMemory();

/* Board specific RDC settings */
BOARD_RdcInit();

BOARD_InitBootPins();
BOARD_BootClockRUN();
BOARD_InitDebugConsole();
```

图 44

（3） 创建 uart 任务，创建成功则开启任务队列，否则打印失败提示后进入死循环。

```
NVIC_SetPriority(DEMO_IRQn, 3);
if (xTaskCreate(uart_task, "Uart_task", configMINIMAL_STACK_SIZE + 100, NULL, uart_task_PRIORITY, NULL) != pdPASS)
{
    PRINTF("Task creation failed!.\r\n");
    while (1)
        ;
}
vTaskStartScheduler();
for (;;)
    ;
```

图 45

（4） 在 uart 任务中，首先从串口接收 4 个字符，存放在 recv_buffer 中。

```
124        error = UART_RTOS_Receive(&handle, recv_buffer, sizeof(recv_buffer), &n);
```

图 46

（5） 判断接收缓冲区是否被占满，占满则挂起任务。

```
125        if (error == kStatus_UART_RxHardwareOverrun)
126        {
127            /* Notify about hardware buffer overrun */
128            if (kStatus_Success !=
129                UART_RTOS_Send(&handle, (uint8_t *)send_hardware_overrun, strlen(send_hardware_overrun)))
130            {
131                vTaskSuspend(NULL);
132            }
133        }
134        if (error == kStatus_UART_RxRingBufferOverrun)
135        {
136            /* Notify about ring buffer overrun */
137            if (kStatus_Success != UART_RTOS_Send(&handle, (uint8_t *)send_ring_overrun, strlen(send_ring_overrun)))
138            {
139                vTaskSuspend(NULL);
140            }
141        }
```

图 47

（6） 将 recv_buffer 中的 4 个字符输出至串口。

```
142        if (n > 0)
143        {
144            /* send back the received data */
145            UART_RTOS_Send(&handle, recv_buffer, n);
146        }
```

图 48

更多帮助

销售邮箱：sales@tronlong.com

技术邮箱：support@tronlong.com

创龙总机：020-8998-6280

技术热线：020-3893-9734

创龙官网：www.tronlong.com

技术论坛：www.51ele.net

官方商城：tronlong.tmall.com